

Assignment 3

Implementing Interfaces and Programming with a Mutable Type

Objective: build a small command-processing system that uses interfaces for behavior, keeps mutation inside a controlled domain type, and remains correct when parsing, validation, ordering, and repeated execution become observable.

1 Task

Create a console application named DroneEnergyControl. It manages three mutable battery units and executes energy commands in the exact order in which they are received.

1.1 Initial battery units

UNIT ID	CAPACITY	INITIAL CHARGE
D1	100	60
D2	80	70
D3	50	10

1.2 Input commands

Use the following data in Program.cs. Do not delete or reorder the lines.

```
string[] rawCommands =
{
    "C001|Charge|D1|-|15",
    "C002|Consume|D2|-|20",
    "C003|Transfer|D1|D3|25",
    "C004|Transfer|D2|D3|20",
    "C005|99|D1|-|5",
    "C003|Consume|D1|-|10",
    "C006|Transfer|D1|D1|5",
    "C007|Charge|D3|-|1",
    "C008|Consume|D3|-|35"
};
```

1.3 Required output

Print one result for every input line, then print the final charge of each unit and the summary. The wording may differ, but the values and classifications must match Section 5.

2 Required behavior

2.1 Mutable domain type

Implement BatteryUnit as a mutable class with the following public shape:

```
public sealed class BatteryUnit
{
    public string Id { get; }
    public decimal Capacity { get; }
    public decimal Charge { get; private set; }

    // Constructor and state-changing methods belong here.
}
```

- Id must be non-empty; Capacity must be greater than zero.
- Initial Charge must be between 0 and Capacity, inclusive.
- External code must not assign Charge directly. Every change goes through methods that preserve the invariants.
- Charging exactly to Capacity and consuming exactly to zero are valid operations.

2.2 Interface and command implementations

Create the interface below and implement it with ChargeCommand, ConsumeCommand, and TransferCommand.

```
public interface IEnergyCommand
{
    string CommandId { get; }

    CommandResult Execute(IReadOnlyDictionary<string, BatteryUnit> units);
}
```

After parsing, the processor must execute commands through IEnergyCommand.Execute. A switch may be used by the parser to construct a command, but a central switch/if on the runtime command type is not allowed for execution.

2.3 Results are data

Represent outcomes with ordinary result objects (for example Applied, Rejected, and InvalidCommand). Do not print from BatteryUnit or command classes. Program.cs is responsible for formatting output.

2.4 Command rules

- Charge adds Amount to SourceId only if the new charge does not exceed Capacity.
- Consume subtracts Amount from SourceId only if the charge does not become negative.
- Transfer moves Amount from SourceId to TargetId only when both changes are valid.
- A transfer to the same unit is rejected.
- A rejected command must not change any battery unit.

3 Parsing, ordering, and state

3.1 Parsing contract

Each line has exactly five pipe-separated fields:

```
commandId|kind|sourceId|targetId|amount
```

- Trim every field. Empty command IDs and unit IDs are invalid.
- Kind is parsed case-insensitively, but only declared enum values are valid.
- Amount uses a dot as the decimal separator, is parsed with `CultureInfo.InvariantCulture`, and must be greater than zero.
- For Charge and Consume, TargetId must be '-'. For Transfer, a non-empty target ID is required; domain checks happen during execution.
- Unit IDs and command IDs are compared with `StringComparer.OrdinalIgnoreCase`.
- An invalid line produces `InvalidCommand` and must not be executed.

```
const NumberStyles AmountStyle =  
    NumberStyles.AllowLeadingSign |  
    NumberStyles.AllowDecimalPoint;  
  
decimal.TryParse(text, AmountStyle,  
    CultureInfo.InvariantCulture, out var amount);
```

3.2 Duplicate protection

A valid command ID may be executed at most once. Check duplicates before calling `Execute`. Because comparison is case-insensitive, `c003` is a duplicate of `C003` and must be rejected without changing state.

3.3 Atomic transfer

Transfer is all-or-nothing: validate the source balance and the target capacity before changing either object. If one check fails, both original charges must remain unchanged.

3.4 Eager execution

`ExecuteAll` must return a completed `IReadOnlyList<CommandResult>` (or `List<CommandResult>`). Do not hide mutation inside a lazy `IEnumerable` produced with `yield return`. Enumerating results twice must never apply commands twice.

```
static IReadOnlyList<CommandResult> ExecuteAll(  
    IEnumerable<string> lines,  
    IReadOnlyDictionary<string, BatteryUnit> units)
```

4 Debugging tasks

Copy the three faulty fragments below into your solution, fix them, and explain each defect in README.md. A replacement with unexplained behavior does not receive debugging credit.

4.1 Undeclared numeric enum

```
static bool TryParseKind(string text, out CommandKind kind)
{
    return Enum.TryParse(text, ignoreCase: true, out kind);
}
```

4.2 Partial mutation during transfer

```
public bool TransferTo(BatteryUnit target, decimal amount)
{
    if (Charge < amount) return false;

    Charge -= amount;
    if (target.Charge + amount > target.Capacity) return false;

    target.Charge += amount;
    return true;
}
```

4.3 Duplicate checked too late

```
var processedIds = new HashSet<string>();

foreach (IEnergyCommand command in commands)
{
    results.Add(command.Execute(units));
    processedIds.Add(command.CommandId);
}
```

4.4 What your explanation must identify

Why the failed transfer can still reduce the source charge, and how to make the operation atomic.

Why duplicate detection must happen before Execute and use a case-insensitive HashSet.

Testing warning: storing another reference to a BatteryUnit is not a snapshot. To verify that a failed transfer changed nothing, save the numeric Charge values before the call and compare the values afterward.

5 Expected results and tests

5.1 Expected result for the provided data

LINE	CLASSIFICATION	REASON / EFFECT
C001	Applied	D1: 60 → 75
C002	Applied	D2: 70 → 50
C003	Applied	D1: 75 → 50; D3: 10 → 35
C004	Rejected	D3 would exceed capacity; neither unit changes
C005	Invalid	99 is not a declared CommandKind
C003	Rejected	Duplicate of C003; no execution
C006	Rejected	Source and target are the same unit
C007	Invalid	Amount must be greater than zero
C008	Applied	D3: 35 → 0

SUMMARY ITEM	EXPECTED VALUE
RAW COMMANDS	9
APPLIED	4
REJECTED VALID COMMANDS	3
INVALID INPUT	2
FINAL CHARGES	D1 = 50; D2 = 50; D3 = 0

5.2 Automated tests

Write at least 12 automated tests. The following nine are mandatory:

- the nine provided lines produce 4 Applied, 3 Rejected, and 2 Invalid results;
- C004 leaves both D2 and D3 unchanged;
- C005 is invalid even though Enum.TryParse can accept numeric enum text;
- c003 is detected as a duplicate of C003 before execution;
- a transfer to the same unit is rejected without mutation;
- a negative or zero amount is invalid;
- charging exactly to Capacity succeeds;
- consuming exactly to zero succeeds;
- enumerating the returned results more than once does not execute commands again.

Choose at least three additional boundary or invalid-input cases of your own.

6 README and submission

6.1 README.md

Answer briefly and in your own words:

1. Which parts of your program are pure, and where is mutation allowed?
2. Why is `IEnergyCommand.Execute` preferable to a central type switch for execution?
3. How does your `TransferTo` implementation guarantee all-or-nothing behavior?
4. Why must duplicate checking happen before `Execute`?
5. Why would `yield return` be dangerous in `ExecuteAll`?
6. Why is a second reference to `BatteryUnit` not a snapshot of its previous state?
7. Which invariants are protected by `BatteryUnit` itself?

6.2 Deliverables

- A compilable .NET console project named `DroneEnergyControl`.
- `README.md` with the required explanations and debugging notes.
- Automated tests in a separate test project.
- Archive name: `IT-25XX_Surname_Name_Assignment3.zip`.

7 Grading

Criterion	Points
Parsing and validation	2
Interfaces and polymorphic dispatch	2
Controlled mutable model and invariants	2
Atomic processing and duplicate protection	2
Debugging explanations and automated tests	2
Oral defense	5
Total	15

7.1 Mandatory grading conditions

1. No oral defense: 0/15.
2. The project does not compile: maximum 5/15.
3. The project compiles but cannot process the provided data: maximum 7/15.
4. A criterion that the student cannot explain during defense receives 0 points.
5. Declaring `IEnergyCommand` but executing via a central runtime-type switch: 0 points for the interface criterion.
6. Public mutation of `Charge` or direct assignment outside `BatteryUnit`: 0 points for the mutable-model criterion.
7. A rejected transfer changes either unit: 0 points for the atomic-processing criterion.
8. Mutation hidden in a lazy sequence, so re-enumeration repeats commands: 0 points for the atomic-processing criterion.
9. Copied code or code generated by AI that the student cannot explain: 0/15.
10. When several limits apply, the lowest maximum grade is used.